UTEC Posgrado

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

ESCUELA DE POSGRADO

Proyecto final de Base de Datos - Parte 1

Movilidad de alto volumen en Nueva York

Viajes de Uber y Lyft

AUTORES

Sebastian Rodrigo García Villacorta
Dante Guillermo Barreto Daza
Clinton Michael Chulluncuy Reynoso
Ronald Ramiro Hilario Orihuela

DOCENTE

Luis Alberto Chicoma Chevez

CURSO

Base de Datos (MCD8008)

Lima — Perú, 2026

Índice

Resumen ejecutivo **1**

1. Análisis del negocio y alcance **2**

1.1. Contexto del negocio 2

1.2. Usuarios, actores, canales y productos 2

1.3. Problema identificado 2

1.4. Objetivo general 2

1.5. Objetivos específicos 3

1.6. Unidad de análisis y límites 3

2. Fuentes de datos, alcance y trazabilidad **3**

2.1. Archivos utilizados 3

2.2. Universo y muestra procesada 4

2.3. Trazabilidad de los lotes 4

2.4. Reglas de calidad 4

3. Modelado Entidad-Relación **5**

3.1. Entidades principales 5

3.2. Cardinalidades 5

3.3. Normalización 6

4. Implementación en PostgreSQL **7**

4.1. Tipos de datos y restricciones 7

4.2. Particionamiento mensual 7

4.3. Carga reproducible e idempotencia 7

5. Consultas SQL orientadas al negocio **8**

5.1. Q01. Volumen mensual por plataforma 8

5.2. Q02. Distribución territorial de las recogidas 9

5.3. Q03. Pares origen-destino más frecuentes 10

5.4. Q04. Espera válida por zona y plataforma . . . 11
5.5. Q05. Variación mensual del promedio diario . . . 12
5.6. Q06. Variación intradiaria en Midtown Center . . . 13
5.7. Q07. Viajes compartidos y accesibilidad . . . 14
5.8. Q08. Componentes económicos . . . 14
5.9. Q09 y Q10. Consultas utilizadas en el benchmark . . . 15
5.10. Q11. Conciliación de la muestra e incidencias . . . 15
5.11. Q12. Ranking mensual de boroughs . . . 16

6. Feature engineering mediante SQL **17**
6.1. Caso de uso y unidad predictiva . . . 18
6.2. Prevención de fuga temporal . . . 19

7. Optimización de consultas e índices **20**
7.1. Diseño experimental . . . 20
7.2. Interpretación de Q09 . . . 22
7.3. Interpretación de Q10 . . . 22

8. Limitaciones, continuidad y conclusiones **24**
8.1. Limitaciones . . . 24
8.2. Continuidad mensual . . . 24
8.3. Conclusiones . . . 24

A. Criterio para las evidencias visuales **25**

B. Procedimiento de reproducción **25**

Índice de figuras

1. Modelo relacional del proyecto: once tablas distribuidas en tres esquemas. . . . 6
2. Creación de tablas, restricciones y particiones en PostgreSQL. 8
3. Resultado de Q01: volumen mensual por plataforma. 9
4. Resultado de Q02: recogidas por distritos y plataforma. 10
5. Top 20 de pares origen-destino del último mes completo. 11
6. Top 20 de espera media válida por zona y plataforma. 12
7. Top 15 de variación mensual del promedio diario. 13
8. Variación horaria en Midtown Center. 13
9. Indicadores de viajes compartidos y vehículos accesibles. 14
10. Evolución de la tarifa base y del pago al conductor. 14
11. Conciliación de lotes e incidencias de calidad. 15
12. Ranking mensual de distritos mediante funciones ventana. 17
13. Ejemplo del conjunto de variables históricas para la zona 161. 20
14. Planes de ejecución antes de crear los índices. 23
15. Planes de ejecución después de crear los índices. 23
16. Índices creados y verificados en PostgreSQL. 24

Índice de tablas

1. Universo disponible y muestra procesada por mes 4
2. Resultados funcionales de las consultas Q09 y Q10 15
3. Variables derivadas para modelado predictivo 19
4. Comparación de planes antes y después de los índices 22

Resumen ejecutivo

El proyecto integra registros *High Volume For-Hire Vehicle* (HVFHV) publicados por la New York City Taxi and Limousine Commission para el periodo comprendido entre enero y mayo de 2026. Los cinco archivos de origen contienen 105 996 114 filas. Debido a restricciones de procesamiento local, se seleccionó una muestra aleatoria reproducible de 1 000 000 de registros por mes, utilizando la semilla 42. De los 5 000 000 de registros procesados, 4 999 931 superaron las reglas de validación y fueron cargados, mientras que 69 fueron rechazados. El informe mantiene explícita la diferencia entre el universo disponible, la muestra procesada y las filas efectivamente cargadas.

La solución fue implementada en PostgreSQL y organiza once tablas distribuidas en los esquemas `core`, `auditoria` y `analytics`. El diseño separa catálogos maestros, control de lotes, viajes, reglas e incidencias de calidad, así como agregados analíticos. La tabla `core.viaje` se encuentra particionada mensualmente y la trazabilidad de cada lote conserva el nombre del archivo, su hash SHA-256 y la conciliación entre filas procesadas, cargadas y rechazadas.

Se desarrollaron doce consultas de negocio, ocho variables históricas para un caso de predicción horaria y dos experimentos de optimización. En el entorno evaluado, Q09 redujo su tiempo de ejecución de 601.535 ms a 0.195 ms y Q10 de 570.161 ms a 0.760 ms, sin modificar los resultados funcionales. Estos valores demuestran el efecto de los índices sobre consultas selectivas dentro del entorno de prueba, pero no deben interpretarse como una garantía universal de rendimiento.

Palabras clave: PostgreSQL, HVFHV, NYC TLC, modelo relacional, calidad de datos, particionamiento, funciones ventana, optimización de consultas.

1. Análisis del negocio y alcance

1.1. Contexto del negocio

El proyecto analiza el proceso de registro de viajes de movilidad bajo demanda operados por plataformas de alto volumen, específicamente Uber y Lyft. El proceso observable comprende la solicitud del servicio, la asignación del vehículo, la llegada del conductor, la recogida, el desplazamiento, la finalización del viaje y el registro de sus componentes económicos.

Cada fila representa un viaje publicado por una base autorizada. Por esta razón, el conjunto de datos no constituye una réplica de los sistemas transaccionales internos de las plataformas ni permite observar las solicitudes que no concluyeron en un viaje registrado.

1.2. Usuarios, actores, canales y productos

Los principales usuarios del servicio son pasajeros y conductores. Sin embargo, los archivos no contienen identificadores personales, por lo que el análisis se realiza exclusivamente a nivel agregado. Los actores modelados son las plataformas, las bases operativas, las zonas de origen y destino y la TLC como entidad publicadora.

Los canales corresponden a las aplicaciones digitales utilizadas para solicitar y gestionar el servicio. Los productos observables se limitan a viajes regulares, viajes compartidos y viajes vinculados con vehículos accesibles para sillas de ruedas.

1.3. Problema identificado

La publicación mensual mediante archivos independientes dificulta la comparación histórica, el control de calidad, la preservación de catálogos y la reproducción de los análisis. Además, el volumen de los archivos exige una estructura física capaz de restringir el acceso a los periodos relevantes y evitar exploraciones innecesarias de la tabla completa.

1.4. Objetivo general

Diseñar e implementar una base de datos relacional en PostgreSQL que integre los registros HVFHV de enero a mayo de 2026, preserve su trazabilidad, controle la calidad de los datos y permita responder preguntas operativas y analíticas mediante consultas SQL reproducibles y optimizadas.

1.5. Objetivos específicos

- Diseñar un modelo relacional normalizado que represente plataformas, bases, zonas, periodos, lotes, viajes e incidencias.
- Implementar restricciones de integridad y particionamiento mensual.
- Conciliar las filas procesadas, cargadas y rechazadas de cada lote.
- Desarrollar consultas de negocio relacionadas con volumen, territorio, rutas, tiempos, accesibilidad y componentes económicos.
- Crear variables históricas mediante funciones ventana, evitando fuga temporal.
- Evaluar el efecto de índices compuestos mediante `EXPLAIN (ANALYZE, BUFFERS)`.

1.6. Unidad de análisis y límites

La unidad transaccional es un viaje publicado, identificado técnicamente mediante la clave compuesta `(mes_fuente, fila_fuente)`. En el informe, el término *demanda* se utiliza como una aproximación operativa basada en recogidas observadas. No representa solicitudes no atendidas, cantidad de usuarios únicos ni cuota integral del mercado de transporte de Nueva York.

Los códigos *Unknown*, *Outside of NYC* y EWR se conservan tal como aparecen en el catálogo oficial. No se reasignan a boroughs distintos porque hacerlo introduciría supuestos no documentados.

2. Fuentes de datos, alcance y trazabilidad

2.1. Archivos utilizados

El estudio considera cinco archivos mensuales HVFHV correspondientes a enero, febrero, marzo, abril y mayo de 2026, además del catálogo `taxi_zone_lookup.csv`. Este último proporciona los nombres de las zonas, boroughs y clases de servicio relacionadas con los identificadores de origen y destino [1, 2].

2.2. Universo y muestra procesada

Tabla 1: Universo disponible y muestra procesada por mes

Mes	Filas de origen	Procesadas	Cargadas	Rechazadas
2026-01	20 940 373	1 000 000	999 967	33
2026-02	19 875 686	1 000 000	999 990	10
2026-03	22 058 358	1 000 000	999 979	21
2026-04	20 995 953	1 000 000	999 999	1
2026-05	22 125 744	1 000 000	999 996	4
Total	105 996 114	5 000 000	4 999 931	69

La muestra permite ejecutar las consultas de forma reproducible en un entorno local. No obstante, los conteos y porcentajes presentados describen la muestra cargada y no el universo completo de 105,9 millones de registros. El modelo puede utilizarse con los archivos completos sin modificar las relaciones principales ni las preguntas de negocio.

2.3. Trazabilidad de los lotes

Cada periodo se registra en `core.lote_ingesta`. La tabla almacena el mes de origen, nombre del archivo, hash SHA-256, cantidad de filas del archivo, filas procesadas, cargadas y rechazadas, estado del lote y fechas de ejecución. El hash permite detectar si un archivo asociado con un mes fue reemplazado por otra versión.

La conciliación satisface la siguiente igualdad:

$$\text{filas cargadas} + \text{filas rechazadas} = \text{filas procesadas}$$

Esta validación demuestra completitud respecto de la muestra seleccionada. No demuestra que se haya procesado el universo completo.

2.4. Reglas de calidad

Las validaciones controlan la plataforma, pertenencia de las zonas al catálogo, secuencia temporal básica, distancia no negativa, duración positiva y correspondencia entre la fecha del viaje y el mes fuente. Los importes negativos son convertidos a `NULL` para evitar interpretarlos como valores económicos válidos o como ceros.

Se registraron 66 240 advertencias de tipo `SOLICITUD_POSTERIOR`. Estos viajes se conservaron

porque pueden resultar válidos para análisis que no dependen del tiempo transcurrido entre solicitud y recogida. Para las consultas de espera se excluyen explícitamente dichos registros y también las esperas superiores a cuatro horas.

3. Modelado Entidad-Relación

3.1. Entidades principales

El modelo contiene once tablas: nueve en `core`, una en `auditoria` y una en `analytics`. Las entidades principales son plataforma, base operativa, borough, zona de servicio, zona taxi, periodo mensual, lote de ingesta, viaje, regla de calidad, incidencia y demanda horaria.

Las particiones mensuales de `core.viaje` son estructuras físicas de la misma entidad y no deben contarse como tablas conceptuales adicionales.

3.2. Cardinalidades

Una plataforma puede registrar muchos viajes y cada viaje pertenece a una plataforma. Una zona puede intervenir en muchos viajes como origen y en muchos viajes como destino. Asimismo, una base puede relacionarse con múltiples viajes como base de despacho o como base de recepción.

La relación conceptual de muchos a muchos entre zonas se resuelve mediante `core.viaje`, que conserva cada ocurrencia con sus atributos temporales, territoriales y económicos. La incidencia de calidad puede no referenciar a un viaje cargado porque una fila rechazada debe conservar trazabilidad aun cuando no ingrese a la tabla transaccional.

En términos de cardinalidad, `core.borough` y `core.zona_servicio` presentan relaciones **1:N** con `core.zona_taxi`: cada zona pertenece a un único borough y a una única clase de servicio. `core.plataforma` y `core.lote_ingesta` se vinculan **1:N** con `core.viaje`. Las relaciones de `core.zona_taxi` y `core.base_operativa` con `core.viaje` también son **1:N** en sus respectivos roles de origen/destino y despacho/recepción; los códigos de base son opcionales en el registro de viaje.

Entre `core.periodo_mensual` y `core.lote_ingesta` la cardinalidad es **1:0..1**: el mes es clave primaria y foránea en el lote, que pertenece a un único periodo. A su vez, periodos, zonas y plataformas se relacionan **1:N** con `analytics.demanda_hora`.

Por su parte, tanto `core.lote_ingesta` como `core.regla_calidad` se relacionan **1:N** con `auditoria.incidencia`. La relación conceptual **N:M** entre zonas de origen y destino se materializa en `core.viaje`: cada registro posee un único origen y destino, pero un mismo par

de zonas puede aparecer en múltiples viajes.

3.3. Normalización

El núcleo relacional satisface la Tercera Forma Normal. La información descriptiva de plataformas, boroughs, zonas y clases de servicio se almacena en catálogos separados, mientras que las medidas del viaje dependen de su clave completa.

La tabla `analytics.demanda_hora` constituye una desnormalización deliberada y reconstruible. Su finalidad es reducir el costo de las consultas horarias y facilitar la creación de variables históricas. El campo JSONB de las incidencias conserva detalle técnico flexible, pero no reemplaza las relaciones ni claves del modelo.

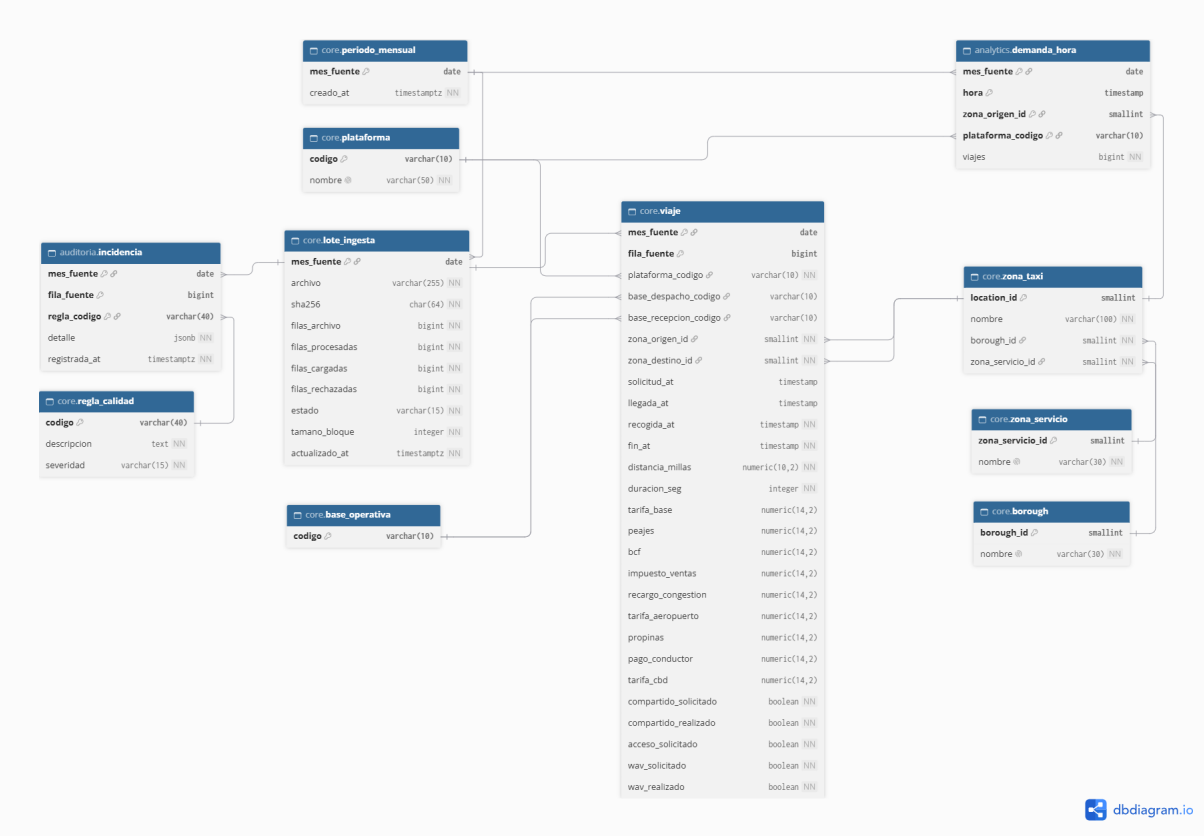


Figura 1: Modelo relacional del proyecto: once tablas distribuidas en tres esquemas.

Diagrama interactivo: Consultar el modelo relacional en dbdiagram.io

4. Implementación en PostgreSQL

4.1. Tipos de datos y restricciones

La implementación emplea `SMALLINT` para identificadores de zona, `BIGINT` para ordinales y conteos, `NUMERIC(14,2)` para importes y `BOOLEAN` para indicadores. Las restricciones `PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `CHECK` y `NOT NULL` protegen la consistencia antes de ejecutar análisis.

4.2. Particionamiento mensual

La tabla `core.viaje` está particionada por rango de `mes_fuente`. Cada partición contiene un mes de información. Cuando una consulta incluye el predicado mensual, PostgreSQL puede descartar las particiones no relacionadas con el periodo solicitado [3].

El particionamiento no elimina por sí solo la necesidad de índices. La poda restringe el conjunto de particiones y el índice reduce posteriormente las filas examinadas dentro de la partición seleccionada.

4.3. Carga reproducible e idempotencia

Los catálogos y lotes utilizan `ON CONFLICT` para evitar duplicaciones durante una repetición controlada. Si el hash de un archivo cambia para un mes ya registrado, la versión nueva debe revisarse y registrarse explícitamente. No corresponde mezclar automáticamente dos archivos distintos bajo el mismo periodo.

	table_schema name	table_name name
1	analytics	demanda_hora
2	auditoria	incidencia
3	core	base_operativa
4	core	borough
5	core	lote_ingesta
6	core	periodo_mensual
7	core	plataforma
8	core	regla_calidad
9	core	viaje
10	core	viaje_2026_01
11	core	viaje_2026_02
12	core	viaje_2026_03
13	core	viaje_2026_04
14	core	viaje_2026_05
15	core	zona_servicio
16	core	zona_taxi

Figura 2: Creación de tablas, restricciones y particiones en PostgreSQL.

5. Consultas SQL orientadas al negocio

Las consultas SQL permiten analizar los principales patrones operativos, territoriales, temporales y económicos de los viajes registrados. Los resultados obtenidos facilitan la evaluación del volumen de viajes, la distribución geográfica de las recogidas, las rutas más frecuentes, los tiempos de espera, los indicadores de accesibilidad y la evolución de los componentes económicos. El análisis se realiza considerando el alcance de la muestra y los criterios de calidad establecidos, con el propósito de obtener conclusiones técnicamente sustentadas y relevantes para la toma de decisiones.

5.1. Q01. Volumen mensual por plataforma

Pregunta: ¿Cuántos viajes de la muestra se registraron por plataforma y mes?

Uber reúne 3 590 362 viajes, equivalentes al 71,81 % de la muestra cargada, mientras que Lyft reúne 1 409 569, equivalentes al 28,19 %. Los conteos brutos deben complementarse con tasas diarias, dado que febrero contiene menos días que enero, marzo y mayo. La participación

describe la composición de la muestra HVFHV y no la cuota de todo el transporte de Nueva York.

	mes text	plataforma character varying (50)	viajes bigint
1	2026-01	Lyft	271486
2	2026-01	Uber	728481
3	2026-02	Lyft	287558
4	2026-02	Uber	712432
5	2026-03	Lyft	276814
6	2026-03	Uber	723165
7	2026-04	Lyft	267650
8	2026-04	Uber	732349
9	2026-05	Lyft	306061
10	2026-05	Uber	693935

Figura 3: Resultado de Q01: volumen mensual por plataforma.

5.2. Q02. Distribución territorial de las recogidas

Pregunta: ¿Cómo se distribuyen las recogidas entre los distritos y las plataformas durante el período analizado?

Manhattan concentra 1 769 318 recogidas, Brooklyn 1 376 386 y Queens 1 094 582. En conjunto representan el 84,81 % de la muestra cargada. La concentración permite identificar territorios con mayor actividad observada, pero no determina sus causas ni representa el lugar de residencia de los pasajeros.

	borough character varying (30)	plataforma character varying (50)	viajes bigint
1	Manhattan	Uber	1281244
2	Brooklyn	Uber	971511
3	Queens	Uber	786858
4	Bronx	Uber	490945
5	Manhattan	Lyft	488074
6	Brooklyn	Lyft	404875
7	Queens	Lyft	307724
8	Bronx	Lyft	186178
9	Staten Island	Uber	59605
10	Staten Island	Lyft	22655
11	Outside of NYC	Uber	198
12	Outside of NYC	Lyft	63
13	EWR	Uber	1

Figura 4: Resultado de Q02: recogidas por distritos y plataforma.

5.3. Q03. Pares origen-destino más frecuentes

Pregunta: ¿Cuáles son los veinte pares origen-destino más frecuentes del último mes completo y cuál es su duración media?

La consulta presenta los veinte pares de zonas con mayor frecuencia durante el último mes completo. East New York - East New York ocupa la primera posición con 3 842 viajes y una duración media de 8,29 minutos. JFK Airport - Outside of NYC registra 3 153 viajes y 61,52 minutos de duración media.

Un par con la misma zona de origen y destino representa desplazamientos internos dentro de su delimitación. La categoría *Outside of NYC* agrupa destinos heterogéneos y no debe interpretarse como una ruta vial específica.

	zona_origen_id smallint	zona_origen character varying (100)	zona_destino_id smallint	zona_destino character varying (100)	viajes bigint	duracion_media_min numeric
1	76	East New York	76	East New York	3842	8.29
2	132	JFK Airport	265	Outside of NYC	3153	61.52
3	138	LaGuardia Airport	265	Outside of NYC	2716	54.68
4	39	Canarsie	39	Canarsie	2299	6.86
5	26	Borough Park	26	Borough Park	1815	9.44
6	61	Crown Heights North	61	Crown Heights North	1641	8.49
7	14	Bay Ridge	14	Bay Ridge	1638	6.47
8	216	South Ozone Park	216	South Ozone Park	1583	6.74
9	95	Forest Hills	95	Forest Hills	1491	7.92
10	7	Astoria	7	Astoria	1433	7.01
11	129	Jackson Heights	129	Jackson Heights	1429	8.37
12	230	Times Sq/Theatre District	265	Outside of NYC	1199	42.82
13	181	Park Slope	181	Park Slope	1169	7.58
14	37	Bushwick South	37	Bushwick South	1142	9.30
15	36	Bushwick North	37	Bushwick South	1118	8.45
16	145	Long Island City/Hunters Po…	145	Long Island City/Hunters Po…	1106	7.06
17	82	Elmhurst	82	Elmhurst	1013	8.27
18	89	Flatbush/Ditmas Park	89	Flatbush/Ditmas Park	1006	8.53
19	37	Bushwick South	36	Bushwick North	1002	8.58
20	42	Central Harlem North	42	Central Harlem North	998	7.33

Figura 5: Top 20 de pares origen-destino del último mes completo.

5.4. Q04. Espera válida por zona y plataforma

Pregunta: ¿Qué zonas y plataformas presentan las mayores esperas medias válidas en el último mes completo?

La consulta excluye solicitudes posteriores a la recogida y esperas superiores a cuatro horas. Asimismo, exige al menos 1 000 viajes por grupo para reducir resultados inestables basados en pocos registros. La evidencia presenta las veinte combinaciones con mayor espera media válida.

Hammels/Arverne - Uber alcanza 8,36 minutos con 1 154 esperas válidas. Le siguen JFK - Lyft con 8,14 minutos y Far Rockaway - Uber con 8,07 minutos. El indicador prioriza grupos para una revisión operativa, pero no atribuye responsabilidad causal a una plataforma o zona.

	zona_origen_id smallint	zona character varying (100)	plataforma character varying (50)	viajes_con_solicitud bigint	esperas_validas bigint	espera_promedio_min numeric
1	117	Hammels/Arverne	Uber	1199	1154	8.36
2	132	JFK Airport	Lyft	4775	4691	8.14
3	86	Far Rockaway	Uber	1519	1472	8.07
4	132	JFK Airport	Uber	11572	11562	7.89
5	138	LaGuardia Airport	Uber	13294	13284	7.77
6	138	LaGuardia Airport	Lyft	7161	7046	7.16
7	203	Rosedale	Uber	1314	1298	6.98
8	23	Bloomfield/Emerson Hill	Uber	1129	1111	6.91
9	235	University Heights/Morris Heig…	Uber	3023	2989	6.70
10	35	Brownsville	Uber	4318	4237	6.69
11	42	Central Harlem North	Uber	5608	5527	6.65
12	259	Woodlawn/Wakefield	Uber	2449	2418	6.64
13	53	College Point	Uber	1131	1114	6.60
14	205	Saint Albans	Uber	2795	2752	6.55
15	217	South Williamsburg	Uber	1174	1154	6.53
16	219	Springfield Gardens South	Uber	1232	1215	6.50
17	72	East Flatbush/Remsen Village	Uber	3592	3547	6.49
18	69	East Concourse/Concourse Vill…	Uber	3619	3588	6.47
19	60	Crotona Park East	Uber	1521	1498	6.41
20	124	Howard Beach	Uber	1105	1090	6.39

Figura 6: Top 20 de espera media válida por zona y plataforma.

5.5. Q05. Variación mensual del promedio diario

Pregunta: ¿Qué zonas registran los mayores incrementos del promedio diario de viajes respecto del mes anterior?

La consulta normaliza los viajes por la cantidad de días del mes, obtiene el período anterior mediante `LAG` y construye el ranking mediante `ROW_NUMBER`. Breezy Point/Fort Tilden/Riis Beach aumenta 82,14 % en febrero y Homecrest 31,92 %.

Los porcentajes elevados pueden originarse en bases pequeñas. Por ello, la variación debe interpretarse junto con los promedios diarios del mes anterior y actual. La consulta presenta las 15 primeras posiciones de cada mes para facilitar su lectura.

	mes_fuente date	zona_origen_id smallint	zona character varying (100)	viajes_diarios numeric	promedio_diario_anterior numeric	variacion_pct numeric	posicion bigint
1	2026-02-01	27	Breezy Point/Fort Tilden/Riis Bea…	1.82	1.00	82.14	1
2	2026-02-01	111	Green-Wood Cemetery	1.64	1.00	64.29	2
3	2026-02-01	253	Willets Point	1.21	0.77	56.85	3
4	2026-02-01	154	Marine Park/Floyd Bennett Field	1.68	1.19	40.64	4
5	2026-02-01	265	Outside of NYC	1.93	1.39	39.04	5
6	2026-02-01	31	Bronx Park	12.43	9.42	31.95	6
7	2026-02-01	123	Homecrest	152.21	115.39	31.92	7
8	2026-02-01	120	Highbridge Park	3.39	2.58	31.47	8
9	2026-02-01	194	Randalls Island	4.50	3.45	30.37	9
10	2026-02-01	166	Morningside Heights	149.86	115.90	29.30	10
11	2026-02-01	59	Crotona Park	4.50	3.48	29.17	11
12	2026-02-01	178	Ocean Parkway South	50.71	39.45	28.55	12
13	2026-02-01	96	Forest Park/Highland Park	4.39	3.45	27.27	13
14	2026-02-01	23	Bloomfield/Emerson Hill	66.25	52.52	26.15	14
15	2026-02-01	157	Maspeth	119.93	95.32	25.81	15

Figura 7: Top 15 de variación mensual del promedio diario.

5.6. Q06. Variación intradiaria en Midtown Center

Pregunta: ¿Cómo varía la demanda horaria por plataforma en Midtown Center durante el día analizado?

La serie del 15 de mayo contiene 48 observaciones, correspondientes a las 24 horas y las dos plataformas. En Uber, la hora 00:00 registra 14 viajes frente a 17 de la hora previa; a la 01:00 se registran 8 frente a 14.

La consulta identifica cambios de corto plazo dentro de un día particular. No es suficiente para establecer un patrón estacional general sin evaluar múltiples fechas comparables.

	hora timestamp without time zone	plataforma_codigo character varying (10)	viajes_objetivo bigint	demanda_1h_previa bigint	cambio_absoluto bigint	cambio_pct numeric
1	2026-05-15 00:00:00	HV0003	14	17	-3	-17.65
2	2026-05-15 00:00:00	HV0005	8	9	-1	-11.11
3	2026-05-15 01:00:00	HV0003	8	14	-6	-42.86
4	2026-05-15 01:00:00	HV0005	3	8	-5	-62.50
5	2026-05-15 02:00:00	HV0003	3	8	-5	-62.50
6	2026-05-15 02:00:00	HV0005	1	3	-2	-66.67
7	2026-05-15 03:00:00	HV0003	4	3	1	33.33
8	2026-05-15 03:00:00	HV0005	0	1	-1	-100.00
9	2026-05-15 04:00:00	HV0003	6	4	2	50.00
10	2026-05-15 04:00:00	HV0005	0	0	0	[null]

Figura 8: Variación horaria en Midtown Center.

5.7. Q07. Viajes compartidos y accesibilidad

Pregunta: ¿Qué proporción de los viajes corresponde a servicios compartidos y qué comportamiento presentan los indicadores de accesibilidad?

En mayo, Uber registró 9 896 viajes compartidos realizados, equivalentes al 1,43 % de sus viajes de ese mes. En contraste, Lyft registró 73 viajes compartidos, que representan el 0,02 % de sus viajes. Estos resultados muestran una mayor participación relativa de los servicios compartidos en Uber durante el período analizado.

Un vehículo WAV (*Wheelchair Accessible Vehicle*) está adaptado para transportar personas usuarias de sillas de ruedas. Los indicadores `wav_solicitado` y `wav_realizado` corresponden, respectivamente, a la solicitud y realización del servicio; al representar eventos distintos, su relación no constituye una tasa directa de atención.

	plataforma character varying (50)	viajes bigint	compartido_solicitado bigint	compartido_realizado bigint	pct_viajes_compartidos numeric	wav_solicitado bigint	wav_realizado bigint
1	Lyft	306061	90	73	0.02	1258	19414
2	Uber	693935	17305	9896	1.43	1861	78508

Figura 9: Indicadores de viajes compartidos y vehículos accesibles.

5.8. Q08. Componentes económicos

Pregunta: ¿Cómo evolucionan la tarifa base y el pago al conductor por plataforma durante el período analizado?

La tarifa base media de Uber aumenta de USD 26.18 en enero a USD 29.50 en mayo. En Lyft aumenta de USD 24.45 a USD 26.90. El pago medio al conductor durante mayo es USD 22.43 para Uber y USD 20.86 para Lyft.

Estas diferencias combinan distancia, ruta, horario y composición de la muestra. No representan rentabilidad de las plataformas ni permiten establecer por sí solas una comparación causal de precios.

	mes_fuente date	plataforma character var	viajes bigint	con_tarifa bigint	tarifa_base_media_ numeric	con_pago bigint	pago_conductor_medio_ numeric	suma_tarifa_base_ numeric	suma_pago_conductor_ numeric
1	2026-01-01	Lyft	271486	271485	24.45	271486	19.15	6638130.65	5198030.41
2	2026-01-01	Uber	728481	727048	26.18	728475	20.11	19030744.08	14647312.93
3	2026-02-01	Lyft	287558	287554	24.44	287558	19.07	7028517.11	5484996.82
4	2026-02-01	Uber	712432	712210	27.50	712432	21.12	19586738.83	15045533.19
5	2026-03-01	Lyft	276814	276813	26.31	276814	19.54	7282573.53	5409242.88
6	2026-03-01	Uber	723165	722921	28.10	723164	21.39	20315583.65	15464879.23
7	2026-04-01	Lyft	267650	267650	26.12	267650	19.81	6990354.79	5300916.18
8	2026-04-01	Uber	732349	732104	27.63	732349	21.27	20225550.46	15579482.66
9	2026-05-01	Lyft	306061	306061	26.90	306061	20.86	8231995.58	6385547.49
10	2026-05-01	Uber	693935	693595	29.50	693935	22.43	20460753.69	15565598.38

Figura 10: Evolución de la tarifa base y del pago al conductor.

5.9. Q09 y Q10. Consultas utilizadas en el benchmark

Pregunta Q09: ¿Cuántos viajes se recogieron en Midtown Center, zona 161, entre las 08:00 y las 09:00 del 15 de enero de 2026?

Pregunta Q10: ¿Cuántos viajes de Lyft se recogieron durante el mismo intervalo, considerando todas las zonas de origen?

Tabla 2: Resultados funcionales de las consultas Q09 y Q10

Consulta	Población evaluada	Viajes
Q09	Midtown Center, zona 161, entre las 08:00 y las 09:00	10
Q10	Lyft, considerando todas las zonas, entre las 08:00 y las 09:00	571

Q09 contabiliza 10 viajes recogidos en Midtown Center entre las 08:00 y 09:00 del 15 de enero. Q10 contabiliza 571 viajes de Lyft dentro del mismo intervalo, sin restringir la zona. Los conteos corresponden a poblaciones diferentes y no deben compararse entre sí.

Su función dentro del informe consiste en comprobar que la optimización física reduzca el tiempo y las filas examinadas sin alterar el resultado lógico de cada consulta.

5.10. Q11. Conciliación de la muestra e incidencias

Pregunta Q11: ¿La muestra procesada concilia correctamente por lote y qué incidencias de calidad fueron registradas?

La pregunta se reformuló para evitar afirmar que se procesaron todas las filas del universo. Los cinco lotes procesaron un millón de filas cada uno. En total, 4 999 931 registros fueron cargados, 69 rechazados y se registraron 66 240 advertencias.

La conciliación demuestra completitud respecto de la muestra seleccionada. Las advertencias no deben sumarse como filas adicionales, porque un viaje cargado puede producir una incidencia de calidad sin ser rechazado.

	mes_fuente [PK] date	estado character varying (15)	filas_archivo bigint	filas_procesadas bigint	filas_cargadas bigint	filas_rechazadas bigint	advertencias bigint	motivos_de_rechazo bigint
1	2026-01-01	COMPLETO	20940373	1000000	999967	33	12767	33
2	2026-02-01	COMPLETO	19875686	1000000	999990	10	14196	10
3	2026-03-01	COMPLETO	22058358	1000000	999979	21	13551	21
4	2026-04-01	COMPLETO	20995953	1000000	999999	1	13034	1
5	2026-05-01	COMPLETO	22125744	1000000	999996	4	12692	4

Figura 11: Conciliación de lotes e incidencias de calidad.

5.11. Q12. Ranking mensual de boroughs

Pregunta Q12: ¿Cómo evoluciona la posición relativa de los distritos según el volumen mensual de recogidas?

El orden de los distritos se mantiene durante los cinco meses: Manhattan, Brooklyn, Queens, Bronx, Staten Island y Outside of NYC. EWR registra un único viaje en enero y ninguna recogida en los meses restantes, por lo que solo aparece en el primer mes del ranking. La estabilidad ordinal no implica que los volúmenes sean constantes. Solo indica que las variaciones mensuales no fueron suficientes para modificar las posiciones del ranking.

	mes_fuente date	borough character varying (30)	viajes bigint	rango bigint	posicion bigint
1	2026-01-01	Manhattan	349339	1	1
2	2026-01-01	Brooklyn	282084	2	2
3	2026-01-01	Queens	219300	3	3
4	2026-01-01	Bronx	133061	4	4
5	2026-01-01	Staten Island	16139	5	5
6	2026-01-01	Outside of NYC	43	6	6
7	2026-01-01	EWR	1	7	7
8	2026-02-01	Manhattan	347705	1	1
9	2026-02-01	Brooklyn	282647	2	2
10	2026-02-01	Queens	218350	3	3
11	2026-02-01	Bronx	134448	4	4
12	2026-02-01	Staten Island	16786	5	5
13	2026-02-01	Outside of NYC	54	6	6
14	2026-03-01	Manhattan	350392	1	1
15	2026-03-01	Brooklyn	274015	2	2
16	2026-03-01	Queens	218272	3	3
17	2026-03-01	Bronx	140278	4	4
18	2026-03-01	Staten Island	16961	5	5
19	2026-03-01	Outside of NYC	61	6	6
20	2026-04-01	Manhattan	363826	1	1
21	2026-04-01	Brooklyn	266536	2	2
22	2026-04-01	Queens	216706	3	3
23	2026-04-01	Bronx	136779	4	4
24	2026-04-01	Staten Island	16105	5	5
25	2026-04-01	Outside of NYC	47	6	6
26	2026-05-01	Manhattan	358056	1	1
27	2026-05-01	Brooklyn	271104	2	2
28	2026-05-01	Queens	221954	3	3
29	2026-05-01	Bronx	132557	4	4
30	2026-05-01	Staten Island	16269	5	5
31	2026-05-01	Outside of NYC	56	6	6

Figura 12: Ranking mensual de distritos mediante funciones ventana.

6. Feature engineering mediante SQL

El proceso de *feature engineering* transforma los datos transaccionales de los viajes en variables explicativas que pueden utilizarse posteriormente en modelos predictivos. En este trabajo, las nuevas variables se construyen directamente mediante SQL, aprovechando funciones de fecha, agregaciones y funciones de ventana.

Este enfoque permite generar un conjunto de datos reproducible, trazable y consistente con la

información almacenada en la base de datos. Además, evita trasladar innecesariamente grandes volúmenes de información hacia herramientas externas antes de realizar las transformaciones principales.

6.1. Caso de uso y unidad predictiva

El caso de uso propuesto consiste en estimar la cantidad de recogidas que se producirán durante la siguiente hora para cada combinación de zona de origen y plataforma. Esta estimación puede utilizarse para anticipar cambios en la demanda, identificar periodos de mayor actividad y apoyar la planificación operativa. La unidad predictiva está definida por la combinación:

```
(hora, zona_origen_id, plataforma_codigo)
```

Cada registro representa, por tanto, la demanda observada en una hora específica, dentro de una zona de origen determinada y para una plataforma concreta. La variable objetivo se denomina `viajes_objetivo` y corresponde al número de viajes iniciados durante dicha hora. Para asegurar la continuidad temporal del conjunto de datos, se construye una malla horaria completa. Esta malla contiene todas las combinaciones posibles de hora, zona de origen y plataforma dentro del periodo analizado, incluso cuando no se registraron viajes. La construcción de esta malla es importante porque permite diferenciar entre los siguientes casos:

1. Una combinación que existe y presenta una demanda igual a cero.
2. Una combinación para la cual no existe suficiente información histórica.
3. Una discontinuidad temporal producida por datos faltantes.

Los rezagos que no disponen de historia suficiente permanecen como valores `NULL`. Estos valores no se sustituyen automáticamente por cero, debido a que un valor nulo representa ausencia de información histórica, mientras que el valor cero representa una demanda observada igual a cero. Confundir ambas situaciones podría introducir información incorrecta en el conjunto de entrenamiento. A partir de la serie horaria se generan variables temporales, rezagos y promedios móviles. Las variables temporales describen el momento en el que ocurre la observación, mientras que los rezagos y promedios móviles resumen el comportamiento reciente de la demanda.

Tabla 3: Variables derivadas para modelado predictivo

Variable	Definición y justificación	Uso potencial
`hora_dia`	Hora entre 0 y 23; captura el patrón intradiario.	Codificación cíclica
`dia_semana`	Día ISO entre 1 y 7.	Estacionalidad semanal
`fin_semana`	Indicador para sábado o domingo.	Segmentación calendario
`demanda_1h_previa`	Viajes registrados una hora antes.	Persistencia corta
`demanda_misma_hora_dia_anterior`	Viajes exactamente 24 horas antes.	Referencia diaria
`demanda_misma_hora_semana_anterior`	Viajes exactamente 168 horas antes.	Referencia semanal
`promedio_3h_previas`	Promedio de las tres horas anteriores.	Suavizado local
`promedio_24h_previas`	Promedio de las 24 horas anteriores.	Nivel diario

Las variables `hora_dia`, `dia_semana` y `fin_semana` permiten representar patrones temporales recurrentes. Por ejemplo, la demanda puede variar entre horas punta y horas de menor actividad, así como entre días laborables y fines de semana. Por otro lado, las variables de rezago permiten utilizar el comportamiento histórico de cada zona y plataforma. El rezago de una hora captura la persistencia inmediata de la demanda, mientras que los rezagos de 24 y 168 horas representan patrones diarios y semanales, respectivamente. Finalmente, los promedios de las tres y 24 horas anteriores reducen el efecto de variaciones puntuales. Estas variables proporcionan una medida más estable del nivel reciente de demanda y pueden mejorar la capacidad predictiva de los modelos.

6.2. Prevención de fuga temporal

La fuga temporal ocurre cuando una variable explicativa incorpora información que todavía no estaría disponible en el momento de realizar la predicción. Este problema puede producir resultados artificialmente favorables durante el entrenamiento y la evaluación del modelo. Para evitarlo, todas las variables históricas se calculan utilizando exclusivamente observaciones anteriores a la hora objetivo. Las ventanas móviles terminan en `1 PRECEDING`, por lo que no incluyen la demanda correspondiente al periodo que se desea predecir [5].

Por ejemplo, para estimar la demanda de las 10:00, el promedio de las tres horas anteriores únicamente debe utilizar la información de las 07:00, 08:00 y 09:00. La demanda de las 10:00 no puede formar parte del cálculo porque representa la variable que se intenta estimar.

De manera complementaria, se valida que la marca temporal asociada a cada rezago coincida exactamente con una, 24 o 168 horas antes de la hora objetivo. Esta comprobación es necesaria porque la función `LAG` recupera una fila anterior según el orden establecido, pero esa fila no

representa necesariamente un intervalo temporal válido cuando existen discontinuidades en la serie.

Por ejemplo, si falta la observación correspondiente a las 09:00, la fila anterior a las 10:00 podría pertenecer a las 08:00. En ese caso, el valor recuperado no debería considerarse un rezago de una hora. La validación de las marcas temporales permite detectar esta situación y conservar el valor como `NULL`.

En consecuencia, la estrategia aplicada permite garantizar que:

- solamente se utilice información disponible antes de la predicción;
- los promedios móviles no incluyan la hora objetivo;
- los rezagos correspondan a intervalos temporales exactos;
- los periodos sin historia suficiente permanezcan como `NULL`;
- no se confunda la ausencia de información con una demanda igual a cero.

Estas medidas fortalecen la validez del conjunto de datos generado y permiten que la evaluación posterior del modelo represente de manera más realista su desempeño sobre observaciones futuras.

	hora timestamp without time zone	zona_origen_id smallint	plataforma_codigo character varying (10)	viajes_objetivo bigint	hora_dia integer	dia_semana integer	fin_semana boolean	demanda_1h_previa bigint	demanda_misma_hora_dia_anterior bigint	demanda_misma_hora_semana_anterior bigint	promedio_3h_previas numeric	promedio_24h_previas numeric
1	2026-05-15 00:00:00	161	HV0003	14	0	5	false	17	13	15	28.67	17.50
2	2026-05-15 00:00:00	161	HV0005	8	0	5	false	9	5	5	12.33	4.79
3	2026-05-15 01:00:00	161	HV0003	8	1	5	false	14	8	9	20.67	17.54
4	2026-05-15 01:00:00	161	HV0005	3	1	5	false	8	1	2	8.67	4.92
5	2026-05-15 02:00:00	161	HV0003	3	2	5	false	8	7	8	13.00	17.54
6	2026-05-15 02:00:00	161	HV0005	1	2	5	false	3	2	0	6.67	5.00
7	2026-05-15 03:00:00	161	HV0003	4	3	5	false	3	3	4	8.33	17.38
8	2026-05-15 03:00:00	161	HV0005	0	3	5	false	1	0	2	4.00	4.96
9	2026-05-15 04:00:00	161	HV0003	6	4	5	false	4	3	3	5.00	17.42
10	2026-05-15 04:00:00	161	HV0005	0	4	5	false	0	1	0	1.33	4.96
11	2026-05-15 05:00:00	161	HV0003	8	5	5	false	6	2	7	4.33	17.54
12	2026-05-15 05:00:00	161	HV0005	1	5	5	false	0	0	0	0.33	4.92
13	2026-05-15 06:00:00	161	HV0003	6	6	5	false	8	6	4	6.00	17.79
14	2026-05-15 06:00:00	161	HV0005	1	6	5	false	1	3	1	0.33	4.96
15	2026-05-15 07:00:00	161	HV0003	3	7	5	false	6	6	9	6.67	17.79
16	2026-05-15 07:00:00	161	HV0005	2	7	5	false	1	4	3	0.67	4.88
17	2026-05-15 08:00:00	161	HV0003	11	8	5	false	3	10	7	5.67	17.67
18	2026-05-15 08:00:00	161	HV0005	0	8	5	false	2	2	0	1.33	4.79
19	2026-05-15 09:00:00	161	HV0003	13	9	5	false	11	13	11	6.67	17.71
20	2026-05-15 09:00:00	161	HV0005	3	9	5	false	0	3	3	1.00	4.71
21	2026-05-15 10:00:00	161	HV0003	13	10	5	false	13	20	12	9.00	17.71
22	2026-05-15 10:00:00	161	HV0005	3	10	5	false	3	2	4	1.67	4.71
23	2026-05-15 11:00:00	161	HV0003	8	11	5	false	13	14	9	12.33	17.42
24	2026-05-15 11:00:00	161	HV0005	5	11	5	false	3	3	6	2.00	4.75
25	2026-05-15 12:00:00	161	HV0003	16	12	5	false	8	12	20	11.33	17.17

Figura 13: Ejemplo del conjunto de variables históricas para la zona 161.

7. Optimización de consultas e índices

7.1. Diseño experimental

Q09 y Q10 fueron seleccionadas por presentar filtros altamente selectivos. En la fase inicial se eliminan únicamente los índices experimentales, se ejecuta `ANALYZE`, se deshabilita temporalmente el paralelismo y se obtienen planes mediante `EXPLAIN (ANALYZE, BUFFERS)`.

Posteriormente se crean índices compuestos cuyos primeros atributos participan en condiciones de igualdad y cuyo último atributo se utiliza en un rango temporal. El mismo predicado se mantiene antes y después, de modo que la comparación evalúe el efecto del acceso físico y no una modificación de la consulta.

En ambas consultas, el filtro `mes_fuente = DATE '2026-01-01'` coincide con los límites de la partición de enero. Las capturas de las dos fases muestran acceso solo a `core.viaje_2026_01`. Esta *poda de particiones* restringe los datos examinados a enero y es independiente del método de acceso elegido dentro de la partición [3].

La opción `BUFFERS` permite complementar los tiempos con lecturas lógicas: `shared hit` identifica bloques que ya estaban en la caché de búferes compartidos y `shared read` contabiliza bloques incorporados a dicha caché, sin implicar necesariamente una lectura física desde disco. En Q09, las capturas muestran el paso de `hit=1018, read=20406` a `hit=10, read=3`; en Q10, de `hit=1050, read=20374` a `hit=570, read=5`. La disminución de bloques consultados respalda la interpretación del cambio de plan, sin atribuir todos los milisegundos exclusivamente al índice [4].

Para interpretar por qué el optimizador estima como convenientes los diferentes accesos, `ANALYZE` actualiza las estadísticas consultables mediante `pg_stats`. En esta vista, `n_distinct` estima la cantidad de valores distintos (o su proporción si es negativo), mientras que `most_common_vals` identifica los valores más frecuentes. En `mes_fuente` ayudan a caracterizar los meses presentes, aunque la poda se determina por los límites de las particiones; en `zona_origen_id` permiten estimar la selectividad del filtro por la zona 161 y comparar el costo de un escaneo secuencial con el de un acceso mediante índice. No se asignan cifras a estas estadísticas porque no se dispone de una salida verificable de `pg_stats` del entorno experimental [4, 6].

Consulta de comprobación estadística (solo lectura):

```
SELECT schemaname, tablename, attname, inherited,
       n_distinct, most_common_vals, most_common_freqs
FROM pg_stats
WHERE schemaname = 'core'
  AND tablename IN ('viaje', 'viaje_2026_01')
  AND attname IN ('mes_fuente', 'zona_origen_id')
ORDER BY tablename, attname, inherited;
```

Tabla 4: Comparación de planes antes y después de los índices

Consulta	Método de acceso	Costo	Filas reales	Ejecución (ms)
Q09 antes	Seq Scan	41 423.39	10	601.535
Q09 después	Bitmap Heap/Index Scan	48.07	10	0.195
Q10 antes	Seq Scan	41 424.41	571	570.161
Q10 después	Index Scan	857.68	571	0.760

7.2. Interpretación de Q09

Antes de crear el índice, PostgreSQL realizó un escaneo secuencial de la partición mensual y descartó 999 957 filas para obtener 10 registros. Después de crear el índice compuesto por zona de origen y fecha de recogida, el optimizador utilizó un *Bitmap Index Scan* junto con un *Bitmap Heap Scan*, lo que permitió localizar de manera selectiva el rango solicitado.

El tiempo de ejecución disminuyó de 601.535 ms a 0.195 ms, lo que representa una mejora aproximada de 3 085 veces. Este resultado corresponde al entorno y a las condiciones específicas de la prueba, por lo que no debe interpretarse como una propiedad fija del índice.

7.3. Interpretación de Q10

Q10 filtra los viajes por plataforma y rango temporal. Antes de crear el índice, PostgreSQL realizó un escaneo secuencial y descartó 999 396 filas para obtener 571 registros. Después de crear el índice compuesto por plataforma y fecha de recogida, el optimizador utilizó un *Index Scan*, lo que permitió realizar un acceso selectivo a los registros solicitados.

El tiempo de ejecución disminuyó de 570.161 ms a 0.760 ms, lo que representa una mejora aproximada de 750 veces. Este resultado corresponde al entorno y a las condiciones específicas de la prueba, por lo que no debe considerarse una propiedad fija del índice.

	QUERY PLAN text
1	Aggregate (cost=41423.38..41423.39 rows=1 width=8) (actual time=601.485..601.486 rows=1 loops=1)
2	Buffers: shared hit=1018 read=20406
3	-> Seq Scan on viaje_2026_01 viaje (cost=0.00..41423.34 rows=17 width=0) (actual time=59.060..601.431 rows=10 loops=1)
4	Filter: ((recogida_at >= '2026-01-15 08:00:00'::timestamp without time zone) AND (recogida_at < '2026-01-15 09:00:00'::timestamp without ti...
5	Rows Removed by Filter: 999957
6	Buffers: shared hit=1018 read=20406
7	Planning Time: 0.417 ms
8	Execution Time: 601.535 ms

	QUERY PLAN text
1	Aggregate (cost=41424.40..41424.41 rows=1 width=8) (actual time=570.139..570.140 rows=1 loops=1)
2	Buffers: shared hit=1050 read=20374
3	-> Seq Scan on viaje_2026_01 viaje (cost=0.00..41423.34 rows=424 width=0) (actual time=0.205..569.524 rows=571 loops=1)
4	Filter: ((recogida_at >= '2026-01-15 08:00:00'::timestamp without time zone) AND (recogida_at < '2026-01-15 09:00:00'::timestamp without ti...
5	Rows Removed by Filter: 999396
6	Buffers: shared hit=1050 read=20374
7	Planning Time: 0.151 ms
8	Execution Time: 570.161 ms

Figura 14: Planes de ejecución antes de crear los índices.

	QUERY PLAN text
1	Aggregate (cost=48.06..48.07 rows=1 width=8) (actual time=0.140..0.141 rows=1 loops=1)
2	Buffers: shared hit=10 read=3
3	-> Bitmap Heap Scan on viaje_2026_01 viaje (cost=4.57..48.04 rows=11 width=0) (actual time=0.122..0.135 rows=10 loops=1)
4	Recheck Cond: ((zona_origen_id = 161) AND (recogida_at >= '2026-01-15 08:00:00'::timestamp without time zone) AND (recogida_at < '2026-...
5	Filter: (mes_fuente = '2026-01-01'::date)
6	Heap Blocks: exact=10
7	Buffers: shared hit=10 read=3
8	-> Bitmap Index Scan on viaje_2026_01_zona_origen_id_recogida_at_idx (cost=0.00..4.56 rows=11 width=0) (actual time=0.112..0.112 rows...
9	Index Cond: ((zona_origen_id = 161) AND (recogida_at >= '2026-01-15 08:00:00'::timestamp without time zone) AND (recogida_at < '2026-...
10	Buffers: shared read=3
11	Planning Time: 0.329 ms
12	Execution Time: 0.195 ms

	QUERY PLAN text
1	Aggregate (cost=857.67..857.68 rows=1 width=8) (actual time=0.731..0.731 rows=1 loops=1)
2	Buffers: shared hit=570 read=5
3	-> Index Scan using viaje_2026_01_plataforma_codigo_recogida_at_idx on viaje_2026_01 viaje (cost=0.42..857.00 rows=268 width=0) (actual ti...
4	Index Cond: (((plataforma_codigo)::text = 'HV0005'::text) AND (recogida_at >= '2026-01-15 08:00:00'::timestamp without time zone) AND (rec...
5	Filter: (mes_fuente = '2026-01-01'::date)
6	Buffers: shared hit=570 read=5
7	Planning Time: 0.208 ms
8	Execution Time: 0.760 ms

Figura 15: Planes de ejecución después de crear los índices.

	esquema name	tabla name	indice name	definicion text
1	analytics	demanda_hora	idx_demanda_hora_zona_hora	CREATE INDEX idx_demanda_hora_zona_hora ON analytics.demanda_hora USING btree (zona_origen_id, hora)
2	analytics	demanda_hora	pk_demanda_hora	CREATE UNIQUE INDEX pk_demanda_hora ON analytics.demanda_hora USING btree (mes_fuente, hora, zona_origen_id, plataforma_codigo)
3	core	viaje	idx_viaje_plataforma_recogida	CREATE INDEX idx_viaje_plataforma_recogida ON ONLY core.viaje USING btree (plataforma_codigo, recogida_at)
4	core	viaje	idx_viaje_zona_recogida	CREATE INDEX idx_viaje_zona_recogida ON ONLY core.viaje USING btree (zona_origen_id, recogida_at)
5	core	viaje	pk_viaje	CREATE UNIQUE INDEX pk_viaje ON ONLY core.viaje USING btree (mes_fuente, fila_fuente)

Figura 16: Índices creados y verificados en PostgreSQL.

8. Limitaciones, continuidad y conclusiones

8.1. Limitaciones

1. Los resultados corresponden a una muestra reproducible de un millón de filas por mes y no al universo completo.
2. Los archivos no contienen identificadores de pasajero o conductor ni solicitudes que no produjeron viajes.
3. Las zonas taxi reemplazan las coordenadas precisas y las categorías especiales agrupan realidades heterogéneas.
4. Las marcas de tiempo no contienen información explícita de zona horaria.
5. Los benchmarks dependen del hardware, caché, configuración, concurrencia y estadísticas del entorno.
6. Los índices evaluados benefician filtros selectivos, pero no necesariamente todas las agregaciones o consultas analíticas.

8.2. Continuidad mensual

Para incorporar un nuevo periodo se debe registrar el mes, crear el lote y la partición, validar el archivo, cargar los viajes e incidencias, reconstruir el agregado horario y actualizar las estadísticas de PostgreSQL. El hash evita mezclar versiones diferentes de un mismo archivo y la conciliación verifica que todas las filas de la muestra tengan un resultado de carga o rechazo.

8.3. Conclusiones

El modelo integra de manera coherente los catálogos, viajes, controles de auditoría y agregados analíticos. La separación entre universo disponible, muestra procesada y filas cargadas evita sobrestimar el alcance de los resultados y fortalece su reproducibilidad.

Las consultas muestran concentración territorial, diferencias entre plataformas, rutas frecuentes, variaciones temporales e indicadores económicos. Cada interpretación se acompaña de sus denominadores, condiciones de validez y limitaciones, evitando atribuciones causales que no pueden sustentarse con los datos disponibles.

Las ocho variables predictivas utilizan exclusivamente información anterior al intervalo objetivo, reduciendo el riesgo de fuga temporal. Los experimentos Q09 y Q10 demuestran que los índices alineados con condiciones de igualdad y rango pueden transformar accesos secuenciales en accesos selectivos, manteniendo sin cambios el resultado lógico.

En conjunto, la solución satisface los requisitos de análisis del negocio, modelado relacional, implementación en PostgreSQL, consultas SQL, feature engineering y optimización. La trazabilidad por lotes, las restricciones de integridad y la documentación de las limitaciones proporcionan un nivel de rigor apropiado para un informe de posgrado.

A. Criterio para las evidencias visuales

Una captura debe demostrar el resultado de una consulta y no convertir el informe en un registro de pantalla. La imagen debe contener el encabezado de las columnas, las filas relevantes y, cuando sea posible, el nombre o comentario que identifica la consulta.

Para conjuntos extensos se recomienda presentar un Top 10, 15 o 20. El resultado completo permanece reproducible en el archivo SQL. No es necesario adjuntar capturas de tablas con cientos de filas ni mostrar columnas que no contribuyen a responder la pregunta de negocio.

En los planes de ejecución se deben visualizar, al menos, el método de acceso, el costo estimado, las filas reales y el tiempo de ejecución. Las capturas anteriores y posteriores deben corresponder exactamente al mismo predicado.

B. Procedimiento de reproducción

1. Crear una base de datos vacía en PostgreSQL 16 o superior.
2. Ejecutar en orden `01_schema.sql`, `02_tables.sql` y `03_data.sql` de la carpeta `SQL Final`.
3. Importar `datos/viaje_2026_01.csv` a `datos/viaje_2026_05.csv` y las incidencias correspondientes.
4. Reconstruir el agregado `analytics.demanda_hora` y consultar la vista de variables históricas.

5. Ejecutar Q01 a Q12 (archivo `04_queries.sql`) y exportar las capturas con los nombres esperados.

6. Ejecutar `05_indexes.sql` para reproducir el benchmark antes y después de crear los índices.

7. Subir a Overleaf este archivo y la carpeta `imagenes/`.

8. Configurar pdfLaTeX como compilador y compilar dos veces.

Referencias

[1] New York City Taxi and Limousine Commission. *TLC Trip Record Data*. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page.

[2] New York City Taxi and Limousine Commission. *Data Dictionary: High Volume FHV Trip Records*. https://home4.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_hvfhs.pdf.

[3] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation: Table Partitioning*. https://www.postgresql.org/docs/16/ddl-partitioning.html.

[4] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation: Using EXPLAIN*. https://www.postgresql.org/docs/16/using-explain.html.

[5] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation: Window Functions*. https://www.postgresql.org/docs/16/functions-window.html.

[6] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation: pg_stats*. https://www.postgresql.org/docs/16/view-pg-stats.html.